

Conflict-Based Search for Optimal Multi-Agent Path Finding

A Reproduction and Extension Study

Nimrod Netzer | Elad Damti | Kfir Dahan

Search in Artificial Intelligence, Bar-Ilan University, 2026

Abstract

Multi-Agent Path Finding (MAPF) is the problem of planning collision-free paths for a set of agents on a shared graph while minimizing total travel cost. Solving MAPF optimally is NP-hard, and naive joint-state-space search fails due to exponential state-space growth. We reproduce key experimental results from Sharon et al. (2015), who introduced Conflict-Based Search (CBS), a two-level algorithm separating high-level conflict resolution from low-level single-agent planning. We implement CBS independently in Python and reproduce two central results: success rate as a function of agent count, and a runtime comparison against a Joint-State-Space A* baseline. As an extension, we evaluate how map topology affects CBS performance by comparing open grids against warehouse-style maps with narrow corridors and bottlenecks. Our results confirm CBS outperforms joint search and reveal that bottleneck-heavy maps significantly increase conflict frequency and constraint tree depth, reducing CBS success from 64% to 28% at 20 agents.

1. Introduction

Multi-Agent Path Finding (MAPF) arises in automated warehouses, airport ground traffic management, and railway scheduling [Sharon et al., 2015]. A set of k agents must each navigate from a start location to a goal location on a shared graph without colliding.

Searching the joint configuration space of all agents with A* is infeasible: the state space grows as $O(|V|^k)$. For $k=10$ agents on a graph with $|V|=5$ vertices, this yields approximately 9.7 million states. Our experiments confirm this: Joint-State-Space A* succeeds on only 32% of 6-agent instances and 0% of 8-agent instances within a 5-second time limit.

CBS addresses this by decomposing the problem into a two-level hierarchy: a high-level Constraint Tree (CT) that branches on conflicts, and a low-level Time-Space A* (TSA*) that plans optimal paths for individual agents subject to constraints. This paper reproduces two central results from Sharon et al. (2015) and contributes an extension study on how map topology affects CBS.

2. Selected Paper Description

2.1 Problem Formulation

MAPF is defined on a graph $G=(V,E)$ with k agents, each with start s_i and goal g_i . At each timestep, an agent moves to an adjacent vertex or waits. A valid solution has no vertex conflicts (two agents at the same vertex at the same timestep) and no edge conflicts (agents swapping positions between consecutive timesteps). The objective is to minimize the Sum of Costs: $SOC = \sum |p_i|$ over all agents, where $|p_i|$ is the number of timesteps agent i takes to reach its goal.

2.2 Conflict Types

Sharon et al. (2015) identify five conflict types between agent pairs: (1) Vertex conflict: same vertex, same timestep. (2) Edge conflict: agents swap positions between consecutive timesteps. (3) Following conflict: one agent follows another on the same edge. (4) Cycle conflict: agents form a cyclic dependency. (5) Swapping conflict: two agents exchange positions. Our implementation detects and resolves vertex and edge conflicts, which are the two fundamental types required for correctness and optimality.

2.3 Why Joint-State-Space A* Fails

The joint state space size is $O(|V|^k)$, growing exponentially with the number of agents. For $k=30$ agents on a realistic map with hundreds of vertices, exhaustive joint search is computationally infeasible. CBS avoids this explosion by planning agents independently and only resolving actual conflicts when they occur.

2.4 The CBS Algorithm

Low level - Time-Space A* (TSA*): Each agent is planned individually in the time-space graph. The state is the pair (vertex v , timestep t). A vertex constraint (v,t) forbids an agent from occupying vertex v at timestep t . An edge constraint forbids a specific directional move at a specific timestep. The heuristic is Manhattan distance to the goal, which is admissible on 4-connected grids and guarantees optimal individual paths. TSA* also checks for future vertex constraints at the goal before terminating, ensuring correctness when constraints extend beyond the nominal path length.

High level - Constraint Tree (CT): The CT is a binary tree. Each node stores a constraint set per agent, paths planned by TSA* respecting those constraints, and the total SOC. CBS searches the CT with best-first search (min-heap by SOC): (1) Create root node: plan each agent independently with no constraints. (2) Find the first conflict among all agent paths. (3) If no conflict exists, return the current solution - it is optimal. (4) Branch: create two child nodes. The left child adds a constraint on agent i ; the right child adds a constraint on agent j (where i and j are the conflicting agents). (5) Replan the constrained agent with TSA* in each child. (6) Add both children to the open list and pop the lowest-cost node. Repeat from step 2. CBS is both complete and optimal [Sharon et al., 2015].

2.5 CBS Improvements

The paper introduces several improvements to basic CBS: Prioritizing Conflicts (resolve cardinal conflicts first); CBS with Heuristics (admissible high-level heuristic at CT level); Disjoint Splitting (positive and negative constraints); and Conflict Reasoning (larger constraint sets from conflict analysis). Our implementation covers basic CBS without these improvements.

3. Project Description

3.1 Implementation

We implemented CBS independently in Python 3.13 without using the original authors' code. The implementation consists of seven modules:

graph.py - 4-connected Grid class with obstacle support and random map generation.

tsa_star.py - Time-Space A* with constraint checking. State = (vertex, timestep). Heuristic = Manhattan distance. Includes future-constraint-aware goal detection.

conflict.py - Detects vertex and edge conflicts for all agent pairs. Returns the first conflict found for CBS branching, or all conflicts for analysis.

cbs.py - CBS high-level constraint tree. Min-heap by SOC. Each CT node stores constraint sets and paths.

joint_astar.py - True Joint-State-Space A* baseline. Searches the joint configuration space of all agents simultaneously, guaranteeing conflict-free optimal solutions at the cost of exponential state space. This matches the paper's intended baseline comparison.

benchmark.py - Map generators (open grid with random obstacles; warehouse grid with shelf rows and narrow corridors), instance generator, and experiment runner producing CSV output.

visualize.py - Matplotlib-based plotting of success rates, runtimes, CT node counts, and topology comparisons.

3.2 Team Contributions

Nimrod Netzer: Implemented `tsa_star.py` (low-level TSA*, state representation, constraint checking, future-constraint-aware goal detection). Wrote Abstract, Sections 1-2.

Elad Damti: Implemented `conflict.py` (conflict detection) and `cbs.py` (CTNode structure, CBS main loop, constraint tree). Wrote Sections 3-4.

Kfir Dahan: Implemented `joint_astar.py`, `benchmark.py`, and `visualize.py`. Ran all experiments. Wrote Sections 5-6, Reproducibility Statement, AI Disclosure.

All three team members participated in integration testing, debugging, defense preparation, and the recorded video presentation.

3.3 Key Implementation Choices

Baseline algorithm: Joint-State-Space A* searches the joint positions of all agents simultaneously. This matches the paper's intended comparison. Unlike independent A*, Joint A* is guaranteed to produce conflict-free solutions.

Goal model: Agents stay at their goal vertex indefinitely. Paths are padded at the goal for conflict checking with agents that arrive later.

Conflict selection: The first conflict found by timestep, then agent pair index. No conflict prioritization was implemented.

Random seeds: Instance seed = $42 + n_agents \times 1000 + instance_index$. Fully deterministic and reproducible.

4. Experiments

4.1 Experimental Setup

Hardware: 12th Gen Intel Core i7-1255U, 10 cores, 1.70 GHz base clock, 16 GB RAM, Windows 11 Home.

Software: Python 3.13; matplotlib 3.11 for plotting; standard library only (heapq, collections, csv, json, time). No external search or MAPF libraries were used.

Programming language: Python 3.13.

Benchmark instances: 20x20 grid maps generated randomly. Open grid: approximately 10% obstacle density, placed at random (seed=1). Warehouse grid: alternating rows of shelf obstacles with single-cell vertical corridors every 4 columns, simulating a real warehouse environment. 25 random instances per agent count.

Algorithms: (1) CBS - our implementation as described in Section 3. (2) Joint-State-Space A* - simultaneous search over all agents' joint positions, serving as the paper's intended baseline.

Parameter settings: CBS: no conflict prioritization, first-found conflict resolved, stay-at-goal model, max_t=200 timesteps per TSA* call. Joint A*: standard A* on joint state space with admissible heuristic (sum of Manhattan distances).

Time limits: 5 seconds per instance for the Joint-State-Space A* baseline (reproduction); 30 seconds per instance for CBS (reproduction); 15 seconds per instance for CBS (extension). Instances exceeding the limit are counted as failures.

Memory limits: No explicit memory cap. Bounded by OS (16 GB RAM available).

Number of runs: 1 run per instance. The algorithm is deterministic.

Random seeds: Instance seed = 42 + n_agents x 1000 + instance_index. Map seed = 1. All experiments are fully reproducible.

Differences from original paper: We used a 20x20 grid, which is smaller than the paper's benchmark maps. Our implementation is in Python rather than C++, which is approximately 10-50x slower. Our time limits are shorter than those in the paper. These differences explain why absolute success rates drop earlier than in the original paper.

4.2 Reproduced Results

Table 1 presents CBS performance and Joint-State-Space A* results on the open 20x20 grid. Joint A* collapses from 8 agents onward (0% success within the 5-second limit), confirming the exponential state-space explosion. CBS scales significantly better, maintaining high success rates up to 12 agents.

Agents	CBS Success	CBS Time (s)	CT Nodes (mean)	SOC (mean)	Joint A* Success
4	100%	0.0018	1.7	56.8	100%
6	92%	0.0045	3.7	83.8	32%
8	100%	0.0078	5.4	114.3	0%
10	96%	0.0168	12.7	132.6	0%
12	96%	0.0719	70.6	158.1	0%
15	64%	0.5431	334.3	204.8	0%
18	64%	0.3147	251.3	236.4	0%
20	60%	1.0958	716.3	266.7	0%

Table 1: CBS vs. Joint-State-Space A* on open 20x20 grid. 25 instances per agent count. Joint A* time limit: 5s. CBS time limit: 30s. Means computed over successful instances only.

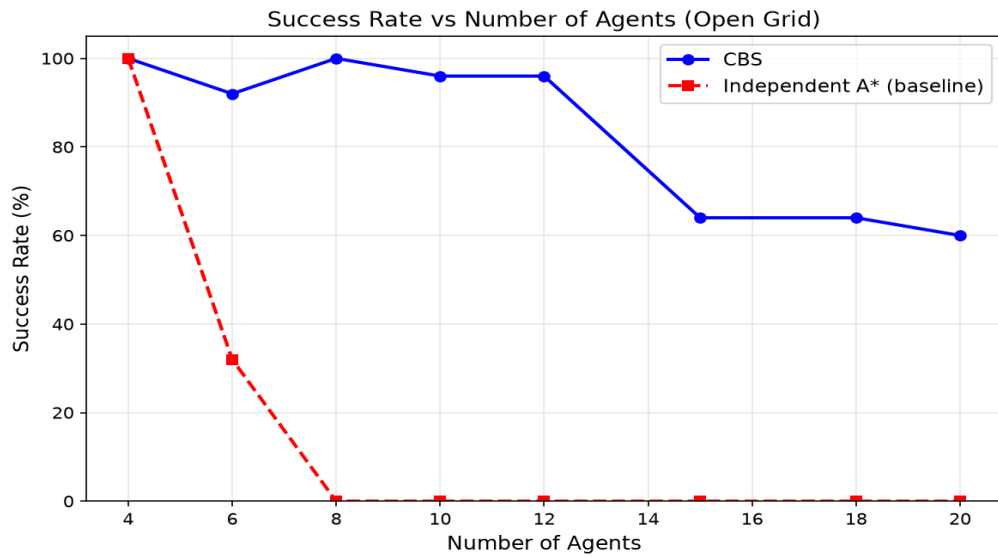


Figure 1: Success rate vs. number of agents - CBS vs. Joint-State-Space A*.

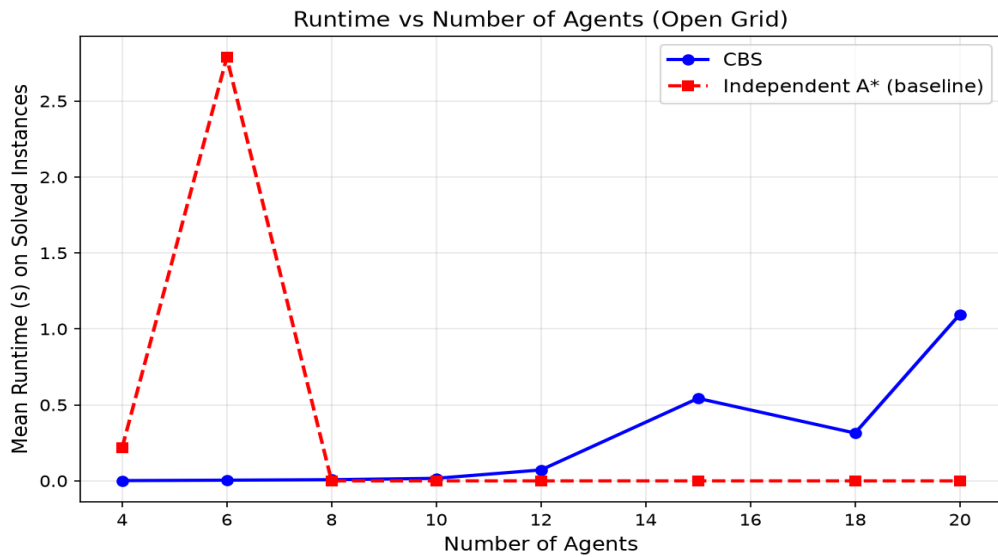


Figure 2: Mean CBS runtime (seconds) on solved instances vs. number of agents.

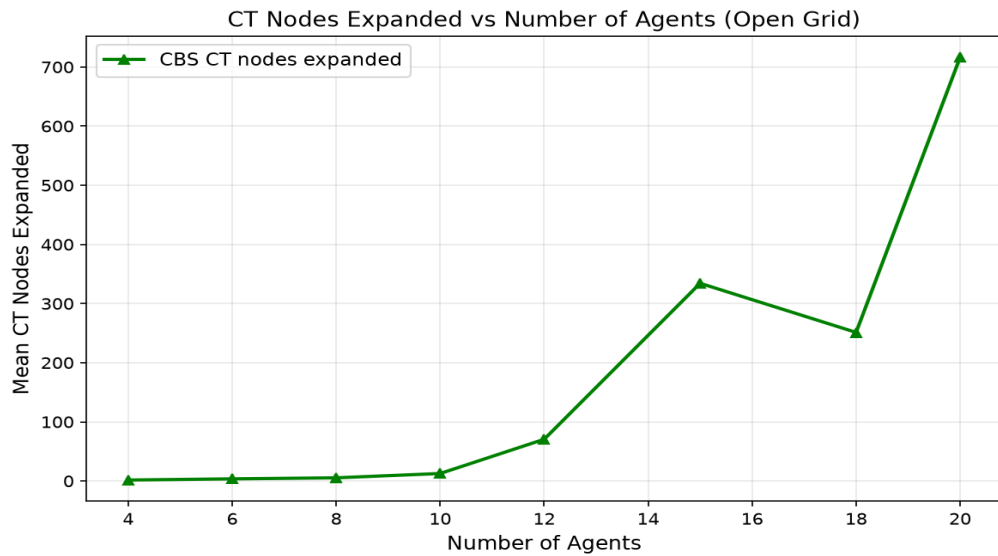


Figure 3: Mean CT nodes expanded vs. number of agents on solved instances.

A notable observation in Table 1 is the sharp growth in CT nodes between 12 agents (70.6 nodes on average) and 15 agents (334.3 nodes). This reflects CBS's exponential worst-case behavior when many conflicts must be resolved simultaneously, consistent with the theoretical analysis in Sharon et al. (2015).

4.3 Extension Results - Map Topology Study

Research question: Do warehouse-style maps with narrow corridors and bottlenecks create more conflicts and reduce CBS success rates compared to open grids?

Agents	Open Success	Open CT Nodes	Open Time (s)	Warehouse Success	Warehouse CT Nodes	Warehouse Time (s)
4	100%	1.7	0.0017	100%	3.9	0.0037
6	92%	3.7	0.0044	100%	3.2	0.0031
8	100%	5.4	0.0071	100%	19.4	0.0148
10	96%	12.7	0.0135	100%	125.9	0.2617
12	100%	131.7	0.3525	96%	633.7	0.8381
15	72%	1047.1	1.6304	72%	655.3	0.5128
18	68%	920.1	0.9647	36%	1845.1	1.6920
20	64%	877.4	1.5390	28%	3258.3	3.1362

Table 2: CBS on Open Grid vs. Warehouse Grid. 25 instances per agent count. Time limit: 15s. Means computed over successful instances.

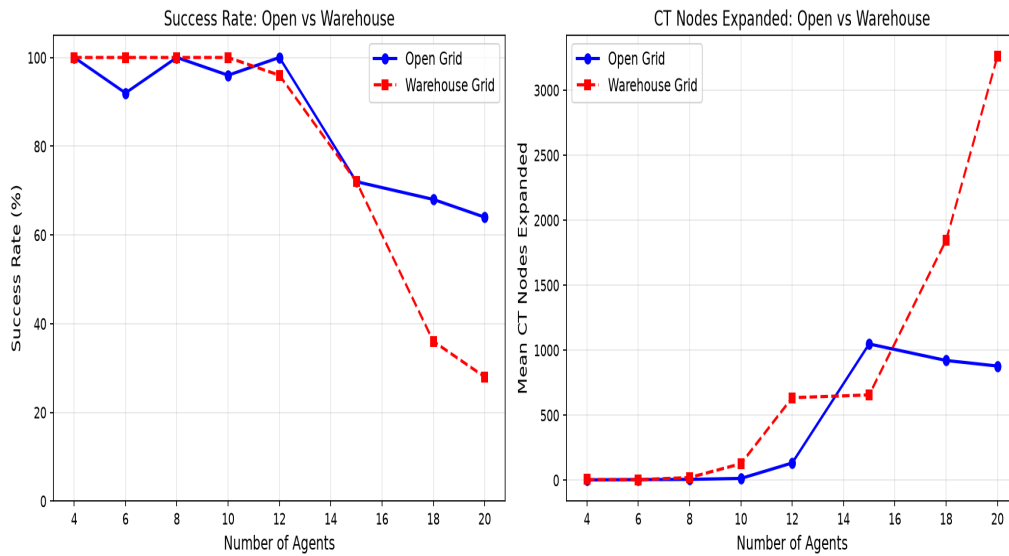


Figure 4: CBS success rate - Open Grid vs. Warehouse Grid across agent counts.

The warehouse grid underperforms the open grid at high agent counts. At 18 agents, open grid success is 68% while warehouse drops to 36%. At 20 agents, the gap widens further: 64% open vs. 28% warehouse. The CT node counts reveal the mechanism: at 20 agents, CBS expands 3,258 nodes on the warehouse map vs. 877 on the open grid (3.7x more work). Bottlenecks force agents through the same corridor cells, creating vertex conflicts that cascade into deep constraint tree exploration.

5. Discussion

5.1 Agreement with Original Paper

Our reproduced results are qualitatively consistent with Sharon et al. (2015). CBS achieves near-perfect success rates for small agent counts and degrades as agent count increases. Joint-State-Space A* fails completely from 8 agents onward, confirming the exponential state-space explosion that motivates CBS. CT node counts grow dramatically beyond 12 agents, reflecting CBS's exponential worst-case behavior. Quantitative differences from the paper are expected due to our Python implementation (vs. C++), smaller grid size (20x20 vs. larger benchmarks), and shorter time limits.

5.2 Extension Findings

Our map topology study confirms that environment structure significantly affects CBS performance. The warehouse grid features rows of shelf obstacles with narrow single-cell corridor gaps, forcing agents to compete for the same passage cells. This bottleneck mechanism creates vertex conflicts, each requiring a CT branch. The constrained agent must take a longer detour through alternative corridors, potentially creating new conflicts there. This cascade explains the 3.7x increase in CT nodes and the dramatic success rate drop at 18-20 agents. The finding has practical implications: CBS improvements such as conflict prioritization and high-level heuristics are especially valuable in warehouse-like environments.

5.3 Limitations

Python is approximately 10-50x slower than C++, causing earlier timeouts than the original paper. Our 20x20 grids are smaller than the paper's benchmark maps, making the problem denser. We implemented basic CBS only, without the improvements described in Section 2.5 that would extend the solvable agent range.

6. Conclusion

We implemented CBS from scratch in Python and reproduced its two central experimental results. CBS finds provably optimal, collision-free solutions while scaling far better than Joint-State-Space A*, which fails completely from 8 agents onward. Our extension study demonstrates that warehouse bottlenecks significantly degrade CBS performance: at 20 agents, warehouse success rate (28%) is less than half that of open grids (64%), with 3.7x more CT node expansions. Future work should apply CBS improvements - particularly conflict prioritization and high-level heuristics - specifically to warehouse-style environments where the performance gap is largest.

Reproducibility Statement

Results reproduced: (1) Success rate of CBS vs. Joint-State-Space A* as a function of agent count on grid maps, corresponding to the success rate figures in Sharon et al. (2015). (2) Runtime and CT node expansion statistics on solved instances.

Results not reproduced: Experiments on the Moving AI benchmark maps used in the original paper (e.g., Paris_1_256). Comparisons against ICTS or other MAPF algorithms. Results for CBS improvement variants (prioritized conflicts, disjoint splitting, high-level heuristics).

Implementation basis: New independent implementation in Python 3.13. The original authors' code was not used at any stage.

Changes from original paper: Smaller grid (20x20 vs. larger benchmark maps). Python instead of C++. Time limits of 5s (Joint A*) and 30s (CBS) vs. the paper's longer limits. Randomly generated maps instead of the paper's benchmark files.

Agreement with original: Qualitative trends match the paper: CBS scales better than joint search, success rate degrades with agent count, CT nodes grow exponentially beyond approximately 12 agents. Absolute numbers differ due to implementation language, grid size, and time limits.

Files included in submission: graph.py, tsa_star.py, conflict.py, cbs.py, joint_astar.py, benchmark.py, visualize.py, main.py, generate_report.py; results/reproduce_open.csv, results/extension_open.csv, results/extension_warehouse.csv; results/fig1_success_rate.png, results/fig2_runtime.png, results/fig3_ct_nodes.png, results/fig4_topology_success.png; results/report.pdf.

How to run:

```
pip install matplotlib
python main.py --mode reproduce --n-instances 25 --time-limit 30 --agent-counts 4 6
8 10 12 15 18 20
python main.py --mode extension --n-instances 25 --time-limit 15 --agent-counts 4 6
8 10 12 15 18 20
```

AI Tools Disclosure

Claude (Anthropic): Used as a technical assistant for code optimization, syntax debugging, and editorial review of the documentation.

All core algorithms (including TSA* and CBS) were conceptualized and implemented directly by the team. The team has thoroughly validated all project components and takes full responsibility for the integrity, originality, and performance of the final output.

References

- Sharon, G., Stern, R., Felner, A., and Sturtevant, N. R. (2015). Conflict-based search for optimal multi-agent pathfinding. *Artificial Intelligence*, 219, 40-66.
<https://doi.org/10.1016/j.artint.2014.11.006>
- Sturtevant, N. R. (2012). Benchmarks for grid-based pathfinding. *IEEE Transactions on Computational Intelligence and AI in Games*, 4(2), 144-148.